Computing and Software for Big Science manuscript No.
(will be inserted by the editor)

Revisiting "What Every Computer Scientist Should Know About Floating-point Arithmetic"

Vincent Lafage

arXiv:2012.02492v1 [math.NA] 4 Dec 2020

Received: 2020

Abstract The differences between the sets in which ideal arithmetics takes place and the sets of floating point numbers are outlined. A set of classical problems in correct numerical evaluation is presented, to increase the awareness of newcomers to the field. A self-defense, prophylactic approach to numerical computation is advocated.

1 Introduction

Precision, and its commoditization, particularly due to the plummeting of its cost and because of its reproducibility, has been the base for our industrial societies since the mid XVIIIth century. The numerical precision went even faster and farther than in mechanical manufacturing.

Thirty years after the paper by Goldberg [1], the global population of developers has arguably grown by a factor of about fifty, and if floating-point was already deemed ubiquitous, the then young IEEE 754 norm is now also ubiquitous, on every desk and in every portable device. Yet "floating-point arithmetic is still considered an esoteric subject by" numerous computer practitioners.

While numerical computations are pervasive nowadays, they may appear for what they are not. Obviously, one needs computers, languages, compilers, and libraries for computing. Furthermore one has to rely on some standard, such as IEEE 754 [2,3,4] and its evolutions [5,6], that quickly became ISO standards [7,8,9]. We note that this standard emerged in a rich environment of incompatible formats, and is now supported on all current major microprocessor architectures. Yet all this is but modeling the numbers. Our familiarity with numbers typically used for computational purpose, *i.e.* the real numbers, may then blur our understanding of floating point numbers.

We will first discuss the common floating point representation of real numbers in a computer and list the differences between the properties of the numbers and the properties of their representation. We will then illustrate common examples where a naive implementation of the computation can lead to unexpectedly large uncertainties. In the next part, we delineate some approaches to understand, contain or mitigate these accuracy problems. We then conclude on the need for a broader awareness about floating point accuracy issue.

2 From real numbers to floating point numbers

Our "intuitive" understanding of numbers, for measurements, payments, arithmetics and higher computations, was developed over years of schooling and essentially builds on decimals which is not only a base-ten positional numeral system, but also the set of all decimal fractions, called $\mathbb{D} = \left\{\frac{n}{10^p}, n \in \mathbb{Z}, p \in \mathbb{N}\right\}$ for decimal. This decimal notation, obvious for us, is rather new, dating back to François Viète in the XVIth century, who introduced systematic use of decimal fractions and modern algebraic notations, and John Napier in the early XVIIth century (improving notations of Simon Stevin) who popularized modern decimal separators (full stop dot as well as comma) to separate the integral part of a number from its fractional part, in his

Vincent Lafage
Université Paris-Saclay, CNRS/IN2P3, IJCLab, 91405, Orsay, France.
E-mail: vincent.lafage@in2p3.fr

Rabdologia (1617). Decimal notation being central for the metric system, its use is widely spread.

But computing with machines rely essentially on binary representations, and computers are using subsets of the binary fractions, called $\mathbb{B} = \left\{\frac{n}{2^p}, n \in \mathbb{Z}, p \in \mathbb{N}\right\}$ for binary.

$\mathbb{B} = \mathbb{Z}[1/2]$ is generated by adjoining $1/2$ to $\mathbb{Z}$, in the same way that $\mathbb{D} = \mathbb{Z}[1/10]$. This construction is a rational extension of ring $\mathbb{Z}$ or localization of the ring of integers with respect to the powers of two. Therefore, algebraically, $(\mathbb{B}, +, *)$ is a ring (like $\mathbb{D}$). As it is not a field, division, as well as all rational functions, will require rounding. As $\mathbb{B}$ (like $\mathbb{D}$) is only a small subset of the algebraic numbers, all roots will require rounding. As $\mathbb{B}$ (like $\mathbb{D}$) is only a small subset of the real numbers ($\mathbb{R}$), all transcendental functions[1] will also require rounding (even elementary ones: trigonometric functions, logarithms and exponential ; and special functions such as Bessel functions, elliptic functions, Riemann ζ function, hypergeometric series, polylogarithms...). As $\mathbb{B}$ is a ring, we get most usual good properties we are used to in mathematics:

- commutativity:
  $\forall x \in \mathbb{B}, \forall y \in \mathbb{B} \quad x + y = y + x,$
  $\forall x \in \mathbb{B}, \forall y \in \mathbb{B} \quad x \times y = y \times x$
- associativity:
  $\forall x \in \mathbb{B}, \forall y \in \mathbb{B}, \forall z \in \mathbb{B} \quad x + (y + z) = (x + y) + z,$
  $\forall x \in \mathbb{B}, \forall y \in \mathbb{B}, \forall z \in \mathbb{B} \quad x \times (y \times z) = (x \times y) \times z$
- distributivity:
  $\forall x \in \mathbb{B}, \forall y \in \mathbb{B}, \forall z \in \mathbb{B} \quad x \times (y + z) = x \times y + x \times z$

Unfortunately, in practice we are limited to elements of $\mathbb{B}$ with a finite representation.

We call these "binary[2] floating point numbers", with a given number of *digits*, in this case *bits*: m bits for their *mantissa* or *significand*, and e bits for their *exponent*. Let's note these sets $\mathbb{F}_{m,e}$: they build an infinite lattice of subsets of $\mathbb{B}$. The size e of the *exponent* and m of the *mantissa* includes their respective sign bits. Furthermore some implementation do not store the most significant bit of the mantissa as it is one by default, then saving one bit for accuracy. Note that, this saving is possible only as we use a base two representation. Most of the time we rely on the hardware embedded $\mathbb{F}_{24,8}$ for single precision and $\mathbb{F}_{53,11}$ for double precision.

[1] as opposed to *algebraic* functions, *i.e.* functions that can be defined as the root of a polynomial with rational coefficients, in particular those involving only algebraic operations: addition, subtraction, multiplication, and division, as well as fractional or rational exponents

[2] We will not deal here with other bases, except to illustrate the cancellation phenomenon and double rounding in the more natural base ten

Topologically, while $\mathbb{B}$ (like $\mathbb{D}$) is dense in $\mathbb{R}$, which would allow for arbitrarily close approximations of real numbers, the finite counterparts of $\mathbb{B}$ (or $\mathbb{D}$) used for floating points, are nowhere dense sets.

Similarly, while $\mathbb{B}$ (like $\mathbb{D}$) are closed under addition and multiplication, their finite counterparts $\mathbb{F}_{m,e}$ is not.

- for multiplication, the number of significant digits of the product is usually the sum of significant digits of factors.
- for addition, the sum will be computed after shifting the less significant term relative to the most significant one, the exact resulting significand getting larger than the original one.

If these results happen to fit into the fixed size, then the computation has been performed exactly, else an `inexact` flag can be raised by the processor and this requires rounding. This is a source of trouble: we cannot catch, or *trap* in the specific case of IEEE 754, all `inexact` exception that would appear in any realistic computation. Most of them even appear in the simplest operation (addition), and then we do not have a choice to handle them: dynamically adaptable precision is possible only with a massive loss of speed, and is useless as soon as roots or transcendental functions[1] are used. These exceptions will appear for all level of precision.

When representing a real number x, one find the two nearest representable floating point numbers: $\mathtt{x}_-$ immediately below x and $\mathtt{x}_+$ immediately above x. Let us note m the middle of the interval $[\mathtt{x}_-, \mathtt{x}_+]$ (a real number that cannot be represented in this floating point format). In order to measure the effect of rounding, one

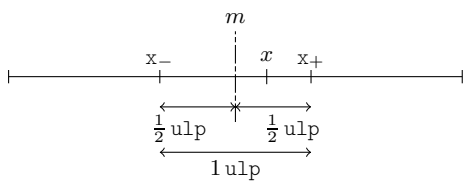

Fig. 1: The basics of rounding

uses the smallest discrepancy between two consecutive floating point numbers, $\mathtt{x}_-$ and $\mathtt{x}_+$: they differ by one unit in the last place (`ulp`): see Fig. 1. So when rounding to the nearest, the difference between the real number and the rounded one should be at most half a `ulp`: this is called *correct rounding*. As this proves difficult, a less strict requirement is to guarantee the rounding within one `ulp` (*i.e.* all bits correct but possibly the last one), which is called *faithful rounding*. In Fig. 1, both $\mathtt{x}_-$ and $\mathtt{x}_+$ are *faithful roundings* of x, but only $\mathtt{x}_+$ is the *correct rounding* of x.

Thus, the behavior of floats is not the same as the behavior of binary numbers. Addition and multiplication inside the set of binary numbers are stable while for floats, it will be inexact most of the time, requiring round-off. From an object oriented point of view, while the floats $\mathbb{F}_{m,e}$ being smaller sets should be subtypes of the binary number as a subset of $\mathbb{B}$, the extra amount of exceptions they bring come in violation of the Liskov substitution principle (LSP) [10,11,12][3].

We must not regard these subsets as behavioral subtypes, which goes against our intuition.

In the same way, while the defined exceptions for single and double are the same, they will not be triggered as often for the highest precision, and in particular for higher precision representation of the same pure number: a product of singles can overflow, where the product of the same values would be computed correctly, and even exactly, with doubles. So we do not get a tower or lattice of inheriting types, the smaller precision deriving from the higher ones, as new behavior (in the form of more exceptions) would emerge from the subtype, in violation of the LSP: in particular, precondition on domain of functions are stricter in the subtype (to prevent overflow or underflow).

Some may consider that subtypes extends the base type, with extra information. In this respect, higher precision would be considered as subtypes of lower precision, extending it with extra digits, and keeping the same methods. Yet, the exception being triggered more often with the lower precision, the higher precision subtype would lead to different behavior (at least better correctness) of the same program.

In order to avoid too many exceptions, particularly exception out of closure, extra elements are added to the set. First, to take overflow into account, $\mathbb{B}$ is affinely extended by signed infinities: $\overline{\mathbb{B}} = \mathbb{B} \cup \{-\infty, +\infty\}$. Then signed zeros $(-0, +0)$ are included for consistency. One more element is convenient to cover for undefined results such as functions on real numbers returning complex numbers, but also indeterminate forms: `NaN` for "Not a Number". With this we turn our number system into a broader representation. We then get another source of `NaN`, as the result of operations involving `NaN`. Which in turns represents different cases so that we may want different `NaN` in our representation to give `NaN` a more specific meaning: these extra informations can be stored in the `NaN` sign bit and in a payload of 23 bits for single precision (or 52 bits for double)[4].

With these extensions the lack of closure of bare float numbers is solved. But this solution comes at the cost of losing associativity and distributivity (a major behavioral shift), that are hard-wired in our ways of thinking about numbers, in the same way the implicit hierarchy of precisions is. Beyond algebraic properties, `NaN` also destroys the total order that exists in $\mathbb{R}$ and $\mathbb{B}$. For low-level programming and optimization, it also breaks the connection between logical equality and binary representation equality.

While the exceptional behaviors are usually apparent, even when replaced by their representations `NaN` or `Infinity`, and the `inexact` exception can be triggered with only half a `ulp` of loss of precision, a much more problematic case will not trigger any more specific exception: "*catastrophic cancellation*" [1], also known as "*loss of significance*" occurs when an operation on two numbers increases relative error more than it increases absolute error, for instance, when subtracting close numbers. As a result of this closeness, the number of significant digits in the outcome is less than expected with the precision. For instance, using here decimal floating points with three digits mantissa for the sake of clarity, when one considers $a = 3.34$ and $b = 3.33$, on the one hand the mathematical difference $a - b = 0.01$ is computed exactly with the given floating point format. This computation, while exact, is a cancellation: one recovers only one significant digit out of three. As this result could not be more exact, this cancellation is said to be *benign*. On the other hand, the mathematical difference of the squares $a^2 - b^2 = 0.0667$ is 6.67×10^{-2} in the given floating point format, but will be evaluated naively as $0.1 = 10^{-1}$ with the rounding. No digit of the floating point result is correct, and this result differ by 50% relative error or 333 units in the last place: this cancellation is *catastrophic*. There is no systematic way to avoid catastrophic cancellation, or to know how many digits have been lost. In particular, catastrophic cancellation will not always lead to a zero, as illustrated with this example.

A corner case among the already pathological behavior of catastrophic cancellation occurs when the absolute value of the computation is below the smallest normal floating point number, *i.e.* 2 to the power of the smallest exponent for the given format: $2^{-128} \sim 0.29 \times 10^{-38}$ for single precision, $2^{-1024} \sim 0.56 \times 10^{-308}$

[3] Liskov's notion of a behavioral subtype defines a notion of substitutability for objects; that is, if S is a subtype of T, then objects of type T in a program may be replaced with objects of type S without altering any of the desirable properties of that program (e.g. correctness).

[4] as is the not well known and even less used case in IEEE 754. Language constructs or libraries to capitalize on this are unfortunately absent. As these payloads are not well known, standard libraries avoid the tedious work of correctly propagating them, which limits even more their usefulness.

for double precision. A strict application of the representation would lead to an underflow exception, turning the result to 0, even if the mathematical result is not strictly null. This could occurs when subtracting close small numbers. To prevent this total loss of relative accuracy, a special representation is included for *subnormal* numbers (formerly known as *denormal* numbers). While these numbers provide algorithms with more stability, dealing with this anomalous representation can slow down computation by as much as 100: this fact brands this part of the norm a shame to high performance computing. A first thing to take into account is that such a slowdown points out a likely improper normalization of the values (see subsection 4.2). A second aspect when subnormal numbers occur, is that extending the precision from single to double is likely to cure the problem at a vastly less cost than redesigning the algorithm or renormalizing the values.

3 Use cases

In this section we will present a collection of use cases where the developers should be aware of the inaccuracies of the floating point computation, and refrain from using the naive approach.

3.1 Converting decimal to binary and binary to decimal

As $\mathbb{B} \subset \mathbb{D}$, every binary floating point number has an exact decimal representation within a finite number of decimal digits. But not conversely: the decimal numbers with which we count our cents may not always get a finite binary representation to the disarray of beginners, the archetypal example being the violation of the mathematical identity $0.1 + 0.2 = 0.3$, whatever the floating-point precision, but more evidently in single precision.

So when, frequently, there is no n digits binary number corresponding exactly to the input decimal number, we round off towards the closest n digits binary number. This requires converting decimal to a precision larger than n binary digits to correctly round it. Hidden in most hardware, below the least significant bit, one finds first a guard bit, a round bit and then a sticky bit: the two first are just two extra bits of precision, while the sticky bit keeps track of all non zero bits lost during the right-shifts of the mantissa. But the decimal to binary conversion is affected by a problem related to the table-maker dilemma (TMD) [13,14] and these three extra bits may not be sufficient to proceed to the correct rounding. The system standard `C` libraries usually provide us with elementary and approximate versions of functions `atof` or `strtof`.

As for the binary to decimal conversion, although every binary floating point number has an exact and finite decimal representation, it is usually not this exact conversion that is used in practice, for instance in a `printf`. This faithful translation would require more digits than the effective precision used for the floating-point format, leading to an absurd overprecision. The decimal number that should be returned is then not the exact one, but the decimal with the shortest expression that stand within one half `ulp` of the binary number. This minimization leads to comparisons and computations with precision higher than the target precision for the routine. The system libraries typically provided a more pragmatic approach, not aiming at correctly rounded but *good enough* results [15].

Yet, correctly rounded conversion routines have existed for the last thirty years [16] (`C` source available on `www.netlib.org/fp`). Faster algorithms even appeared [17]. But they did not seem to make it in our compilers and system or language libraries. While the compilers have gone a long way to evaluate the literal and named constants in extra precision, and that mitigates the problem, the bulk of the conversions comes from decimal data input/output: as long as these are not commoditized as language construct, these aspects of round-off are often considered for experts.

This kind of round-off can pile-up in loop of conversions (from one program to another, for instance when storing intermediate results in decimal format). As a consequence, one should avoid decimal storage as much as possible and prefer binary storage, not only for performance, but also for precision. Moreover, the number of bits for accurately encoding a given number of decimal digits is not the naive estimation: it has been known for over fifty years [18], that the 24 bits of single precision are not enough for an accurate representation of 7 decimal digits, while $24 \log_{10}(2) > 7$. Hence caution is needed when estimating the required precision, usually on number of decimal grounds.

3.2 Double rounding

The classical issue of double rounding occurs when using two different precisions: a high precision and a lower one. The exact result is first rounded to the high precision, then to the lower one. For instance with base 10 floating points, we will consider a mantissa of 2 digits for the high precision and 1 digit for the lower one: with an exact result of 3.47, the rounding to the high precision gives 3.5, and the rounding to the lower precision gives 4, whereas the direct rounding of the exact

result to the lower precision is 2. Double rounding happens when computing in extended precision (such as the hardware implementation of $\mathbb{F}_{64,16}$) that one converts to the smaller working precision: even if the extended precision has been correctly rounded (within half a `ulp`), the rounding to working precision may lead only to a faithful rounding (within one `ulp`). The correct solution is addressed in Ref. [19]. In particular, while double correct rounding is not always correct, the double faithful rounding is faithful.

3.3 Difference of two squares

The difference of two squares $a^2 - b^2$ seems innocuous, and if the units of the program have been chosen correctly, we can easily avoid the overflow risk (however it is enough to trigger this overflow that a or b be of order of 10^{19} in single precision instead of 10^{154} in double precision). The real issue is, when the two terms are close, in the possibility to exhibit "*catastrophic cancellation*" [1], also known as "*loss of significance*". These cancellation occurs in an operation (usually a subtraction) on previously truncated operands, for instance, the product of other operands (here, squares). The numerical result then has only a limited precision compared to that with which the operand computations were made. Even the sign of the result can be wrong.

A contrario the same difference on exact operands leads to a benign cancellation.

Using double precision is usually the way to mitigate this effect.

The best solution is to use a factorized formulation of the computation $a^2 - b^2 = (a - b) \cdot (a + b)$. The remaining subtraction can be the cause of "*benign cancellation*" at most. In this case factorization used to provide a speed gain (a single multiplication instead of two) on top of the precision gain[5]. On contemporary architectures, the speed gain has disappeared, as multiplication is now as fast as addition. In this simple case, the factorization is trivial, but in the expression of the area of a triangle, or the volume of a tetrahedron, the work required is arduous. More generally, in the determinantal expressions common in geometry, there is no guarantee of the existence of these factorizations, and when they exist such results seems to be found by sheer luck.

[5] technically, when a and b are of different magnitude, the non-factorized version suffers one less rounding error, therefore is more precise, but without affecting the meaning of the result

3.4 Summation

Another well known algorithm among the community is Monte-Carlo integration: basically, it is a loop sampling a configuration space, then accumulating a given contribution as function of the sampled point. We therefore proceed to add fluctuating values in a growing accumulator. The more one accumulates, the more the accuracy of the individual contribution is blurred.

Ideally, one should sort the contributions from the smallest to the largest (for contributions with the same sign) to be sure to lose as little precision as possible. One could also apply a "divide and conquer" algorithm such as pairwise summation [20]. Another solution is to accumulate in extended arithmetic, or to use the summation algorithm of W. Kahan [21] or other compensated sums [22,23,24,20,25,26].

Let us specify this essential aspect of sums which is the accumulation of truncation error. As this error will statistically be distributed uniformly as much upwards as downwards, one can assimilate it to a random walk, and in the hypothesis of terms of comparable size, to expect for the sum of N terms to a resulting absolute uncertainty on the total of the order of $\mathcal{O}\left(\sqrt{N}\right)$. This does not seem alarming since the associated relative uncertainty will be in $\mathcal{O}\left(1/\sqrt{N}\right)$. This approach is however naive since the absolute error of truncation of a sum is like the relative error compared to the largest term *i.e.* the accumulator. The relative error varies like $\mathcal{O}\left(\epsilon\sqrt{N}\right)$. Comparatively, pairwise summation has a much tamer relative error varying like $\mathcal{O}\left(\epsilon\sqrt{\log_2 N}\right)$. As a matter of fact, the naive addition of 4×10^6 contributions in a half precision accumulator causes a relative uncertainty of 100 %, and in the case of a single precision accumulator, it reduces the precision within four significant digits at best (if each contribution is correctly calculated to the accuracy of the machine and correctly rounded). The average Monte Carlo can easily produce such a four million events sampling.

We can also combine these mild (and all the more treacherous) losses of precision with catastrophic cancellation. This is, for instance, the case of series falling under the Leibniz's test for alternating series: a sum of terms of alternating signs, of zero limit at infinity and of decreasing absolute values. The convergence of these series is mathematically guaranteed, and the uncertainty on the partial sum is bounded from above by the absolute value of the first neglected term in truncating the serie. Each pair of successive terms of these series is thus concerned with a potentially disastrous compensation. The latest can be overcome in the case

where an analytical expression of terms exists, but there is no general solution for purely numerical expressions.

3.5 Quadratic formula

Let's start with the famous quadratic equation

$$ax^2 + bx + c = 0, \quad \text{where} \quad a \neq 0, \tag{1}$$

$$\text{with} \quad \Delta = b^2 - 4ac, \tag{2}$$

its corresponding discriminant, we note, for $\Delta > 0$, its two exact solutions:

$$x_\pm = \frac{-b \pm \sqrt{\Delta}}{2a}. \tag{3}$$

We can identify two possibilities for "*catastrophic cancellation*": the first, in expressing the discriminant, and the second in the compensation between $-b$ and $\sqrt{\Delta}$. The latter can be overcome reformulating the roots as:

$$q = -b - \operatorname{sgn}(b)\sqrt{\Delta} = -\operatorname{sgn}(b)\left(|b| + \sqrt{\Delta}\right) \tag{4}$$

$$x_1 = \frac{q}{2a}, \tag{5}$$

$$x_2 = \frac{2c}{q} = \frac{c}{ax_1}. \tag{6}$$

q is then always a sum of two like-sign terms, without catastrophic cancellation. We thus obtain an accurate expression without execution branching. On the other hand, avoiding execution branching blurs the link between the two pairs of roots, and a test might be necessary to match x_1 to x_+ or x_-.

A solution for taming the catastrophic cancellation in the discriminant will be presented in Subsection 4.4.

3.6 Polynomial evaluation

It is well known that polynomial function should not be evaluated in the naive way: on the one hand, because of the computational cost of all the exponentiation, on the other hand because of the accuracy loss it represents. As we have seen in Subsection 3.3, when a factored out form exists, it had better be exploited. In addition, when factorization is not known, the best approach is Horner-Ruffini method [27]:

$$\begin{aligned} p(x) &= a_0 + a_1 x + \cdots + a_{n-1}x^{n-1} + a_n x^n \\ &= a_0 + x\Big(a_1 + x\big(\cdots + x(a_{n-1} + x\, a_n)\cdots\big)\Big) \end{aligned} \tag{7}$$

It is not only a gain in speed, through the saving of operations that it achieves, but also in accuracy, partly for the same reason, and more importantly it is a guarantee of stability of the result and safety against intermediate overshoots, particularly in the case of simple precision. It is even possible to increase the evaluation speed [28].

The democratization of "multiply-accumulate" machine instructions (`fma`) combining addition and multiplication (see Subsection 4.4), leads us to strongly seek their use in this case [29].

Furthermore, a polynomial being a sum (see Subsection 3.4), compensation summation techniques, such as the summation algorithm of W. Kahan [21], must be taken into account.

3.7 Variance estimate

The computation of variance estimator is a canonical example of "*catastrophic cancellation*" in that it is a difference of squares. Let us consider the variance of a sample of N individuals.

$$\sigma^2 = \overline{(x - \bar{x})^2} = \overline{(x^2)} - \bar{x}^2 = \frac{\sum_{i=1}^N x_i^2 - \frac{\left(\sum_{i=1}^N x_i\right)^2}{N}}{N-1}. \tag{8}$$

A golden rule for students used to be "*never compute the variance with single precision*"... Because the variance so computed could become negative, it appears abruptly when one extract the square root to obtain the standard deviation. As evaluation occurs more and more online, consequences can be dire if proper safeguard exception handling is not provided.

Certainly, a two-pass approach, when possible, allows to stick with single precision. But one can also take advantage of Welford [30] one pass online algorithm. The cost for accuracy then comes as an extra division for each iteration loop.

The problem also appears when evaluating covariance matrices and in evaluating centered moments of higher orders.

3.8 Area of a triangle

The formula of Heron of Alexandria, allows to calculate the area S of any triangle knowing only the lengths a, b and c of its three edges:

$$S = \sqrt{p(p-a)(p-b)(p-c)} \tag{9}$$

with $p = \frac{a+b+c}{2}$ the half perimeter. This nice symmetrical formula exhibits an instability during the numerical computation, which appears for a needle-like triangle, *i.e.* a triangle of which one edge is of particularly small dimension compared with the others (confrontation of small and large values).

By labelling the names of edges so that $a > b > c$, and by reorganizing the terms to optimize the quantities added or subtracted, W. Kahan proposed a more

stable formula for S in Ref. [31]:

$$\frac{1}{4}\sqrt{[a+(b+c)]\ [c-(a-b)]\ [c+(a-b)]\ [a+(b-c)]} \tag{10}$$

While its apparent symmetry is lost, this formula is much more robust than the naive one. He similarly provided a stable formula for the computation of the volume of a tetrahedron based on a non trivial factorization [32].

These calculations may seem hazy textbook case: they are in fact only the expression with small dimensions ($n = 2$, $n = 3$) of factors which intervene in more general geometric calculations, such as the relativistic phase space volume [33]. These factors turn out to be determinants, notoriously delicate to evaluate precisely if one proceeds naively[6]. For instance, if we study the production of intermediate bosons and electrons, their square masses are in a ratio of 10^{10} and certain kinematic factors will be analogous to the surface of a triangle with edges in such a ratio.

3.9 Complex Arithmetic and Analysis

Complex numbers are a powerful tool for the physicist and the engineer. Unfortunately, they are often presented as the simplest example of Cartesian coordinates, which often gives rise to high school-like implementations[7] which are mathematically correct but numerically fragile. These expressions are notably liable to involve intermediate overflow although their result is representable (complex modulus, complex division and even complex multiplication). For instance, when naively evaluating the modulus of a single precision complex number with the largest component greater than 2×10^{19}, the intermediate sum of squares will overflow while the resulting modulus is representable as a single precision floating point number. Other languages integrate complex numbers better, such as `Fortran`, `Ada` and `D`, and even better, `C99`.

As for complex analysis, particularly when evaluating radiative corrections in particle physics, but also when modeling 2D irrotational fluid flows, the subtleties of managing the zero sign in the vicinity of the cuts associated with connection points, can lead to results which are not trivially false [36]. It is therefore important to use established libraries to benefit from guaranteed values, for example for one-loop functions in radiative corrections [37].

3.10 Function evaluation

Function evaluation often start with a range reduction: trigonometric functions are first linked to their expression in the range $[0; \frac{\pi}{4}]$, and building upon the binary representation of the argument, square root over $[\frac{\sqrt{2}}{2}; \sqrt{2}]$, exponential in the range $[0; \ln 2]$, logarithm over $[-\sqrt{2}; \sqrt{2}]$... This apparently trivial transformation can in itself require extra computations to be really accurate. But before algorithms such as power series expansion or Newton-like iteration can be used efficiently, an extra step needs to be taken, as these reduced intervals are already too large for quick convergence. This is commonly achieved through lookup tables (LUT).

3.11 Function Minimization

This almost daily activity in our community, seems well under control. If we look more closely, in the vicinity of its minimum, the studied function is the sum of the minimum value and of a quadratic form (Taylor's theorem). With this form's coefficients close to unity, a relative displacement of $\mathcal{O}(h)$ around the position x_0 of the minimum will generate a relative displacement of $\mathcal{O}(h^2)$ of the function to minimize. However, the smallest relative measurable displacement being bounded by the *machine epsilon* $\epsilon = 2^{-m}$, the variation δx_0 which will not generate a perceptible difference will be $\mathcal{O}(\sqrt{\epsilon})$, half the machine precision. Therefore if we calculate the function with double precision, the uncertainty on the position of the minimum corresponds to the single precision. If we now evaluate the function to minimize in single precision, the position of the minimum will only have the resolution of the half precision. Moreover if we estimate the function by means of the half precision in vogue, the position will be known with less than two significant figures. This half precision being supposed to serve the training in *machine learning*, which rely essentially on minimization, using half precision casts some doubts on the accuracy of the results.

Some studies have tested the consequences of using half precision in training neural networks, including the use of mixed precision with single precision accumulators (see Refs. [38,39] or fixed point number instead of floating point numbers (see Refs. [40,41]). These precisions are sufficient for training and running the network. The question of the accuracy of a neural network reproducing a numerical function is yet open.

[6] Multivariate polynomial in nature, while relying on "simple" arithmetics, these multi-linear alternating form are the best place to exhibit massive cancellation. Computation examples defeating even quadruple precision rely on these higher degree polynomial cancellation (see Ref. [34,35])

[7] see comments such as `// XXX: This is a grammar school implementation.` in the `complex` header of `g++`

4 Some approaches to control floating point uncertainty

Now cautious that our prejudice will work against us, we need some way to compute nevertheless.

4.1 Condition number and stability

As we have seen, some computations have a risk of developping large relative uncertainties even with precise input. In this subsection, we will attempt to quantify the amplification factor between input uncertainty and output uncertainty, and to discriminate the part inherent to the function under study, called the condition number, from the part caused by the evaluation algorithm.

The condition number κ inherent to a mathematical function f evaluated at argument x with approximate value x_a compares the relative uncertainty on $f(x)$ to the relative uncertainty on the argument. It takes the form:

$$\kappa\left(x\right) = \frac{\left|\frac{f(x_a)-f(x)}{f(x)}\right|}{\left|\frac{x_a-x}{x}\right|} = \frac{\left|\frac{f(x_a)-f(x)}{(x_a-x)}\right|}{\left|\frac{f(x)}{x}\right|} \sim \left|\frac{xf'(x)}{f(x)}\right| \qquad (11)$$

κ is also known as the absolute value of the *elasticity* of the function f. Note that κ is dimensionless. This simple expression reveal that even the most elementary function is prone to large uncertainties: considering the addition of a constant c ($f : x \to x + c$), the corresponding condition number $\kappa = \frac{x}{x+c}$ exhibits a singular behavior around $x = -c$, while being tame on the vast majority of value: next to the cancellation point, relative error on the result will be far larger than relative error on the argument. This particular example illustrates that catastrophic cancellation is amplified by a large condition number even before rounding takes place.

Beyond this formal definition, a more concrete consequence of the condition number will help us measure its value as a tool: as the smallest relative uncertainty achievable with a given precision is of half a `ulp`, a neatly computed parameter of a function f will produce a relative uncertainty of $\frac{\kappa}{2}$ `ulp` in the result of f. Hence $\log_{10} \kappa$ is the number of digits of accuracy lost in this computation, and $\log_2 \kappa$ is the number of bits of accuracy lost in this computation.

Moreover, the composition of functions will lead to a net condition number equal to the product of component functions condition numbers. The 23 bits of single precision can then succumb to repeatedly composed evaluation of a function with as moderate a condition number as 2.

This points out the need for different levels of cautiousness in the evaluation strategy of a function, depending on the argument: some subdomains of the definition domain do not exhibit large condition number, others need more care. From the definition, one can easily demonstrate that only power laws $x \to C \times x^n$ (with C and n some real constants) have a constant condition number $\forall x$, $\kappa\left(x\right) = n$. Hence even functions with a nice unique expression may need piecewise evaluation. Beside lacking of aesthetic appeal, this casuistry has a clear impact on performance as the required tests potentially lead to branch misses. Moreover, these tests come on top on those needed for checking that the argument is inside the function's domain of definition.

The condition number of the problem reveals that the precision of the computation may have to be larger than the target accuracy.

Note that until this point, we have considered only exact mathematical expression: the condition number $\kappa\left(x\right)$ does not consider approximations and rounding. Further uncertainties come from the numerical algorithm used to evaluate the expression: approximation, for instance truncation of a power or Fourier series expansion, as well as the rounding, on each term and on the sum. The condition number and extra uncertainties contribute to define an overall *stability* of the algorithm under small perturbations. These concepts are as old as computers [42] but deserve to be given a larger exposure: these were for instance not described in Ref. [1], despite the numerous examples of this paper being developped around singular points for their respective condition number.

Once the singular values of the condition number is found (most of the time simple zeros of function f will be singular for its condition number κ), an *asymptotic analysis* of f around these singular values must be performed to assess the more stable expression: for instance, considering $f = x \to \ln x$, one finds that $\kappa\left(x\right) = \frac{1}{\ln x}$, with singular value $x = 1$, which brings our attention to the expression of f around 1, $f\left(x = 1 + h\right) = \ln\left(1 + h\right)$ for which the condition number becomes:

$$\kappa\left(h\right) = \frac{h}{\left(1+h\right)\ln\left(1+h\right)} \underset{h\to 0}{\sim} \frac{1}{\left(1+h\right)} \qquad (12)$$

which is no more singular for h near 0 (but is singular for h near -1). Yet as the expression involves an indetermination, it needs to be evaluated with care as described in Ref. [1]. Nowadays, the mathematical libraries provide us with `log1p`.

In real situations, the condition number must be extended beyond univariate function by using an arbitrary norm instead of absolute value, and one must take into account the uncertainties on physical constants as

possibly greater than half a `ulp` (as mathematical constants have zero for their condition number).

4.2 Nondimensionalization

Scientists often compute to solve equations or simulate phenomena where variables and parameters have specific units and physical dimensions, following dimensional analysis. Dimensional homogeneity requires that all analytic functions and their arguments must be dimensionless, as these functions can be expressed as an infinite power series of the argument and all terms must have the same dimension. Usage is to extract the *characteristic units* of the problem at hand, and use these for *rescaling* the dimensional quantities to dimensionless ones (Vaschy-Buckingham-Bertrand π theorem). This choice of units allows one to simplify the equation, with parameters both fewer and dimensionless (such as the Reynolds number in hydrodynamics, among others). Beside this aspect, useful to understand the problem, it also proves beneficial numerically as the dimensionless variables are of the order of unity, where the density of representable floating points is the highest (and where the overflow and underflow are as far as possible).

4.3 Interval arithmetic

If one would like a higher precision than the native types of the processor, there are different solutions such as double-double precision, quadruple precision, arbitrary precision, rational representations...

An important alternative is to represent each variable by a pair of floating point numbers representing the bounds of an interval ensuring that it contains the true value. Numbers arithmetic then becomes interval arithmetic, which is more than twice as expensive to evaluate, but provides mathematical certainties. This approach can be traced back to Archimedes and appeared quickly but discreetly in the history of computers. Beyond the calculation of each terminal, it is advisable to round the lower limit towards $-\infty$ and the upper limit towards $+\infty$ and this switching of the control register of the FPU considerably slows down the execution. A number of implementations bypass this problem by systematically subtracting ϵ from the lower bound and adding ϵ to the upper bound. As harmless as this extra ϵ may seem, it feeds the natural demon of interval arithmetic, which constantly increases the size of intervals. The result is then certainly fair, but useless. The recent agreement on a standard will somewhat facilitate the use of the intervals [43,44]. However, it is not enough to couple a library [45,46] to its program to automatically benefit from interval calculation. Worse, beyond the transformation of types, literal constants and, more importantly, input-output, these are the expressions and algorithms that must be radically revised to curb the expansion of the resulting intervals. This technique has already been used for a thesis in the context of calculating uncertainties for data acquisitions [47].

4.4 Good news on accuracy front

Since 2008 [5] the IEEE 754 standard takes into account the operations `fma` "*Fused Multiply and Add*" which allows one to perform only a rounding instead of two on the combination of a multiplication and an addition.

This new operation primarily affects two major algorithms: the scalar product [25] (and therefore the products of matrices but also convolutions) and the evaluation of polynomials by the Horner-Ruffini method.

These two algorithms, based on sums, are intended to be coupled with compensation techniques such as the summation algorithm of Kahan [48,49,50].

`fma` can also be used for Error-Free Transformation (EFT) of arithmetics: given one operation $\circ$ and two floating point numbers a and b, the rounded operation $a \circ b$ would produce an approximate result s, but the corresponding EFT will produce a sum of two floating points values $s + e$ corresponding to the exact result. This capacity is relevant to use when computing a reduced discriminant $\Delta' = b'^2 - ac$ riddled with cancellation when $b'^2 \sim ac$. The EFT for the multiplication has two steps: the most significant result is computed with a simple, approximate (rounded) product $a*b$, also noted $a \otimes b$ distinct from the exact product $a \times b$, and the error term is evaluated with a `fma` as follows:

$$a \times b = a \otimes b + \texttt{fma}(a, b, -a \otimes b) \tag{13}$$

One then apply this EFT to the two terms in $\Delta' = b'^2 - ac$ and if there is a cancellation between the most significant terms of the EFT, adding the difference of the least significant terms will provide for the missing accuracy.

4.5 Stochastic Arithmetic

Beyond these self-defense and stability validation approaches, we can use more recent environments for assessing digital precision. These execution environments induce small disturbances at each calculation stage and thus make it possible to measure the fluctuation of

the result. Here the emphasis is about stochastic arithmetic. This technique is akin to Monte Carlo. The disturbance induced in the computation can appear in the form of either a random noise added to the computation or a random change of mode of rounding of the results. These environments are `CADNA` [51,52,53,54,55], `Verificarlo` [56,57] and `Verrou` [58,59,60,61] which can be used with `valgrind` [62,63,64].

The `Verrou` environment allows, in addition to its stochastic mode, to force all rounding of the calculation in an unconventional direction instead of the nearest rounding: towards $-\infty$, towards $+\infty$, towards 0, and even to test the rounding off "as far as possible", a good way to get an idea of the resilience of ones numerical code. These programs allow an instrumentation of the code which reveals numerical weaknesses at runtime. Their implementation has an impact on the performance of the code, and must be taken into account in the time budget for testing and optimization.

4.6 Double precision as an insurance

The previous examples have shed a light on numerical computation that is far from common expectation: floating-point computing is not straightforward at all, while expressions like "number crunching" evoke blind use of power. In fact, while a mathematical formula looks like a monolithic, deep-frozen archive of theoretical truth, its practical evaluation often requires to be split between different domains and regimes, each relying on mathematically identical but numerically different expressions. This casuistry may feel uncomfortable for the beginner, but it is also irritating to the optimization expert: as pipelining and vectorization are the modern basis of computing performance, tests in the innermost loops of library are shunned. In the same way, cache-misses induced by lookup table are also a pain for performance computing.

All fields of engineering usually involve "*factors of safety*" to provide for unexpected situation. The fields where the materials are known the better can afford to minimize these factors of safety: concrete walls or elevator cables can be oversized with security factors of order twenty, but planes have to stay within a security factor of two: this can only be achieved through a deep and thoroughly documented knowledge of the material at hand. One might see the traditional recourse to double precision as such a factor of safety. While most computations will not be used in real life with more than single precision, intermediate result may require double precision to achieve single accuracy. For end user to rely only on single precision, and possibly benefit on its gain in speed, they need a much higher level of test and documentation of their algorithms and their library implementation.

4.7 Mixed precision

As mentioned in Subsection 3.11, the algebraic intensity of machine learning computations and the best use of both memory size and bandwidth had the community turn to accuracies lower than single precision. Namely half-precision, either the IEEE compliant `binary16`, or `FP16` [5], or the `bfloat16` of Google Brain [65]. `FP16` having a very restricted range (magnitude up to 65 536), one often use a single precision accumulator while multiplying half precision variable.

Besides machine learning, dense and sparse linear algebra, as well as Fast Fourier Transform are studied with mixed precision in Ref. [66].

4.8 Software engineering

The major programming paradigms prevents us from dangerous practices through discipline: structured programming forbids the use of `goto`, modular programming prevents global variables, functional programming forbids mutations, strong-typing prevents implicit cast of type, polymorphism discipline the indirect transfer of control of pointers to functions, *etc.* We should be looking for what to forbid to achieve a safer floating-point environment: let us note that it has not to rely on a specific syntax or language, but more on good practices.

The first and most well known approach is that tests for strict equalities should likely be banned (topological aspect of the representation), but it is only the most elementary defense.

To coin Robert Martin [67] "A good architecture allows you to defer framework decisions". The choice of the precision is such a framework decision. In order to defer this choice, *generic programming* is the paradigm of choice. The easiest way to implement it, is using simple preprocessing of source or `C` style `typedef` to specify at compilation time a generic floating-point type with a given precision. Unfortunately, the precision of the floating-point variables does not guarantee in itself the accuracy of the result. Hence the full generic paradigm is required here, allowing to rewrite the function differently for the various precision if needed.

Unit testing and Test Driven Development may also look beneficial to the numerical computation community, but the amount of testing required is rather large.

5 Conclusion

"*The purpose of computing is insight, not numbers.*" (Richard Wesley Hamming, Turing Award 1968)

Experimental systematic error usually represent a large part of the discussion in a science paper or thesis. However numerical systematic error are rarely discussed. The propagation of numerical errors along the computational process needs to be given attention as part of the systematic error study.

Floating-point numbers are a computer abstraction for real numbers. As we have shown in this paper, this abstraction does not protect us completely from implementation details: to coin Ref. [68,69] floating-point numbers are a *leaky abstraction*.

Fifteen years after the computer power growth is no more being driven by the processor frequency, in order to achieve the expectation on science projects, we face the need for keeping the computing power growing, and some in the field consider moving to less precise computation to execute it faster.

Admittedly double precision is not double accuracy. Floating-point precision is a mean to achieve floating-point accuracy, the goal of our computation, that has to be asserted beforehands. The problem stands in computations larger than the ones we can follow, where not only the accumulation of round-off errors, but the existence of numerical singularities hidden in the more and more complex domain spaces of these computation hinders our confidence in floating points computations. In the same way the software engineering has dealt with the growing complexity of modern programs over seventy years, through a set of disciplines enforced by language paradigms as well as helper tools, floating point computing must be understood as the difficult field it is, deserving also its set of good practices and tools.

As W. Kahan reminds us in the conclusion of Ref. [70], we still have to rely on the usual strategies: different compilers, and if possible different computers (not only a different hardware, but a different architecture, and possibly a different manufacturer), different libraries and different runtime environments also matter. Using binary format data storage from end to end is also a good approach, separating the data model from the data view.

The self-defense techniques suggest us to develop our codes in generic precision, instead of setting from start a fixed precision. The relative safety provided by double precision is dismissed by teams ready for more aggressive approach, particularly with large amount of data, naively confusing precision with accuracy. An empirical, down to earth approach, comparing results of different precisions, should help decide on the precision. Unfortunately, conducting such tests is not as simple as it seems [71].

Acknowledgements We gratefully acknowledge the stimulating discussions we had with Amel Korichi, Hadrien Grasland, Pierre Aubert and Philippe Gauron.

References

1. D. Goldberg, ACM Comput. Surv. **23**(1), 5, Mar. 1991. doi:10.1145/103162.103163
2. W. Kahan, J.F. Palmer, SIGNUM Newsl. **14**(si-2), 13, Oct. 1979. doi:10.1145/1057520.1057522
3. IEEE Standard for Binary Floating-Point Arithmetic. Tech. rep., Oct 1985. doi:10.1109/IEEESTD.1985.82928
4. IEEE Standard for Radix-Independent Floating-Point Arithmetic. Tech. rep., Oct 1987. doi:10.1109/IEEESTD.1987.81037
5. IEEE Standard for Floating-Point Arithmetic. Tech. rep., Aug 2008. doi:10.1109/IEEESTD.2008.4610935
6. IEEE Standard for Floating-Point Arithmetic. Tech. rep., July 2019. doi:10.1109/IEEESTD.2019.8766229
7. Binary floating-point arithmetic for microprocessor systems. Standard, International Organization for Standardization, Geneva, CH, Feb. 1989. URL `https://www.iso.org/standard/19706.html`
8. Information technology — Microprocessor Systems — Floating-Point arithmetic. Standard, International Organization for Standardization, Geneva, CH, Jun. 2011. URL `https://www.iso.org/standard/57469.html`
9. Information technology — Microprocessor Systems — Floating-Point arithmetic. Standard, International Organization for Standardization, Geneva, CH, May 2020. URL `https://www.iso.org/standard/80985.html`
10. B. Liskov, in *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages and Applications (Addendum)* (Association for Computing Machinery, New York, NY, USA, 1987), OOPSLA '87, p. 17–34. doi:10.1145/62138.62141
11. B. Liskov, J.M. Wing, in *ECOOP' 93 — Object-Oriented Programming*, ed. by O.M. Nierstrasz (Springer Berlin Heidelberg, Berlin, Heidelberg, 1993), pp. 118–141
12. B.H. Liskov, J.M. Wing, ACM Trans. Program. Lang. Syst. **16**(6), 1811–1841, Nov. 1994. doi:10.1145/197320.197383. URL `https://doi.org/10.1145/197320.197383`
13. A. Castiel, V. Lefèvre, P. Zimmermann, Interstices 2004. URL `https://hal.inria.fr/inria-00000567`. `http://interstices.info/display.jsp?id=c_5936`. See also `http://www.vinc17.org/research/slides/epao2001-03.pdf`
14. C. Daramy, D. Defour, F. de Dinechin, J.M. Muller, in *Advanced Signal Processing Algorithms, Architectures, and Implementations XIII*, vol. 5205, ed. by F.T. Luk. International Society for Optics and Photonics (SPIE, 2003), vol. 5205, pp. 458 – 464. doi:10.1117/12.505591
15. R. Regan. Quick and Dirty Floating-Point to Decimal Conversion. `https://www.exploringbinary.com/quick-and-dirty-floating-point-to-decimal-conversion`, Nov. 2010
16. D.M. Gay, Correctly Rounded Binary-Decimal and Decimal-Binary Conversions. Tech. Rep. 90-10, Numerical Analysis Manuscript AT&T Bell Laboratories, Murray Hill, New Jersey 07974, Nov. 1990. URL `https://ampl.com/REFS/rounding.pdf`

17. F. Loitsch, SIGPLAN Not. **45**(6), 233, Jun. 2010. doi:10.1145/1809028.1806623
18. I.B. Goldberg, Commun. ACM **10**(2), 105–106, Feb. 1967. doi:10.1145/363067.363112
19. S. Boldo, G. Melquiond, When double rounding is odd. Tech. Rep. Research Report No 2004-48, École Normale Supérieure Lyon, Nov. 2004. URL http://www.ens-lyon.fr/LIP/Pub/Rapports/RR/RR2004/RR2004-48.pdf
20. N.J. Higham, SIAM Journal on Scientific Computing **14**(4), 783, 1993. doi:10.1137/0914050
21. W. Kahan, Commun. ACM **8**(1), 40, Jan. 1965. doi:10.1145/363707.363723. http://doi.acm.org/10.1145/363707.363723
22. J. Mcnamee, ACM SIGSAM Bulletin **38**, 1, 03 2004. doi:10.1145/980175.980177
23. T.O. Espelid, SIAM Review **37**(4), 603, 1995. doi:10.1137/1037130
24. I. Babuška, in *IFIP Congress (1)*, vol. 68 (1968), vol. 68, pp. 11–23
25. T. Ogita, S.M. Rump, S. Oishi, SIAM Journal on Scientific Computing **26**(6), 1955, 2005. doi:10.1137/030601818
26. S.M. Rump, T. Ogita, S. Oishi, SIAM Journal on Scientific Computing **31**(2), 1269, 2009. doi:10.1137/07068816X
27. W.G. Horner, Philosophical Transactions of the Royal Society of London **109**, 308, 1819. doi:10.2307/107508
28. D.E. Knuth, Commun. ACM **5**(12), 595, Dec. 1962. doi:10.1145/355580.369074
29. S. Graillat, P. Langlois, N. Louvet, in *Proceedings of the 2006 ACM Symposium on Applied Computing* (ACM, New York, NY, USA, 2006), SAC '06, pp. 1323–1327. doi:10.1145/1141277.1141585
30. B.P. Welford, Technometrics **4**(3), 419, 1962. doi:10.1080/00401706.1962.10490022. URL https://www.tandfonline.com/doi/abs/10.1080/00401706.1962.10490022
31. W. Kahan, 2014. URL https://people.eecs.berkeley.edu/~wkahan/Triangle.pdf. https://people.eecs.berkeley.edu/~wkahan/Triangle.pdf
32. W. Kahan, 2012. URL https://people.eecs.berkeley.edu/~wkahan/VtetLang.pdf. https://people.eecs.berkeley.edu/~wkahan/VtetLang.pdf
33. E. Byckling, K. Kajantie, 1973
34. S.M. Rump, in *Reliability in computing*, ed. by R.E. Moore (Elsevier, 1988), pp. 109–126. doi:https://doi.org/10.1016/B978-0-12-505630-4.50012-2. URL http://www.sciencedirect.com/science/article/pii/B9780125056304500122
35. E. Loh, G.W. Walster, Reliable Computing **8**(3), 245, 2002
36. W. Kahan, J.W. Thomas, Augmenting a programming language with complex arithmetic. Tech. Rep. UCB/CSD-92-667, EECS Department, University of California, Berkeley, Dec 1991. URL http://www2.eecs.berkeley.edu/Pubs/TechRpts/1991/6127.html
37. G.J. van Oldenborgh, Comput. Phys. Commun. **66**, 1, 1991. doi:10.1016/0010-4655(91)90002-3
38. M. Courbariaux, Y. Bengio, J.P. David, in *Advances in neural information processing systems* (2015), pp. 3123–3131. URL https://arxiv.org/abs/1412.7024. Accepted as a workshop contribution at ICLR 2015
39. P. Micikevicius, S. Narang, J. Alben, G.F. Diamos, E. Elsen, D. García, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, H. Wu, CoRR **abs/1710.03740**, 2017. URL http://arxiv.org/abs/1710.03740
40. S. Gupta, A. Agrawal, K. Gopalakrishnan, P. Narayanan, in *International Conference on Machine Learning* (2015), pp. 1737–1746
41. Y. Choi, M. El-Khamy, J. Lee, CoRR **abs/1809.00095**, 2018. URL http://arxiv.org/abs/1809.00095
42. A.M. Turing, The Quarterly Journal of Mechanics and Applied Mathematics **1**(1), 287, 01 1948. doi:10.1093/qjmam/1.1.287
43. IEEE Standard for Interval Arithmetic. Tech. rep., June 2015. doi:10.1109/IEEESTD.2015.7140721. https://ieeexplore.ieee.org/document/7140721
44. IEEE Standard for Interval Arithmetic (Simplified). Tech. rep., Jan 2018. doi:10.1109/IEEESTD.2018.8277144
45. *2014 IEEE Conference on Norbert Wiener in the 21st Century (21CW)* (Wiley-IEEE Computer Society Press, 2014). doi:10.1109/NORBERT.2014.6893849
46. M. Nehmeier, in *2014 IEEE Conference on Norbert Wiener in the 21st Century (21CW)* [45], pp. 1–6. doi:10.1109/NORBERT.2014.6893854. https://github.com/nehmeier/libieeep1788 Its main focus is the correctness and not the performance of the implementation.
47. F. Hautot, Cartographie topographique et radiologique 3D en temps réel : acquisition, traitement, fusion des données et gestion des incertitudes. Ph.D. thesis, Université Paris Saclay, 2017
48. S. Graillat, P. Langlois, N. Louvet. FMA Implementations of the Compensated Horner Scheme, Jan. 2006. URL https://perso.univ-perp.fr/langlois/slides/dag06_sl.pdf. Reliable Implementation of Real Number Algorithms: Theory and Practice, Dagstuhl Seminar
49. P. Langlois, N. Louvet. Operator Dependant Compensated Algorithms, May 2007. URL http://perso.ens-lyon.fr/nicolas.louvet/LaLo07b.pdf
50. S. Graillat, P. Langlois, N. Louvet. Accurate dot products with FMA, Jul. 2006. URL http://rnc7.loria.fr/louvet_poster.pdf
51. N. Scott, F. Jézéquel, C. Denis, J.M. Chesneaux, Computer Physics Communications **176**(8), 507, 2007. doi:https://doi.org/10.1016/j.cpc.2007.01.005
52. F. Jézéquel, J.M. Chesneaux, Computer Physics Communications **178**(12), 933, 2008. doi:https://doi.org/10.1016/j.cpc.2008.02.003
53. F. Jézéquel, J.M. Chesneaux, J.L. Lamotte, Computer Physics Communications **181**(11), 1927, 2010. doi:https://doi.org/10.1016/j.cpc.2010.07.012
54. J.L. Lamotte, J.M. Chesneaux, F. Jézéquel, Computer Physics Communications **181**(11), 1925, 2010. doi:https://doi.org/10.1016/j.cpc.2010.07.006
55. N. Scott, F. Jézéquel, C. Denis, J.M. Chesneaux. The CADNA library, 2007. URL http://cadna.lip6.fr/
56. C. Denis, P.D.O. Castro, E. Petit. A tool for automatic Montecarlo Arithmetic analysis. URL https://github.com/verificarlo/verificarlo
57. C. Denis, P.D.O. Castro, E. Petit. Verificarlo: checking floating point accuracy through monte carlo arithmetic, Sep. 2015
58. F. Févotte, B. Lathuilière, SCAN 2016 p. 47, 2016
59. F. Févotte, B. Lathuilière, in *13e colloque national en calcul des structures* (Université Paris-Saclay, Giens, Var, France, 2017). URL https://hal.archives-ouvertes.fr/hal-01908466
60. F. Févotte, B. Lathuilière, VERROU: Assessing Floating-Point Accuracy Without Recompiling, Oct. 2016. URL https://hal.archives-ouvertes.fr/hal-01383417. Working paper or preprint
61. F. Févotte, B. Lathuilière. Verrou: a floating-point rounding errors checker. URL http://edf-hpc.github.io/verrou/vr-manual.html. Verrou is french for "lock"
62. URL http://www.valgrind.org/

63. N. Nethercote, J. Seward, Electronic Notes in Theoretical Computer Science **89**(2), 44 , 2003. doi:10.1016/S1571-0661(04)81042-9. RV '2003, Run-time Verification (Satellite Workshop of CAV '03)
64. N. Nethercote, J. Seward, SIGPLAN Not. **42**(6), 89–100, Jun. 2007. doi:10.1145/1273442.1250746
65. D.D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D.T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen, J. Yang, J. Park, A. Heinecke, E. Georganas, S. Srinivasan, A. Kundu, M. Smelyanskiy, B. Kaul, P. Dubey, CoRR **abs/1905.12322**, 2019. URL http://arxiv.org/abs/1905.12322
66. A. Abdelfattah, H. Anzt, E.G. Boman, E. Carson, T. Cojean, J. Dongarra, M. Gates, T. Grützmacher, N.J. Higham, S. Li, N. Lindquist, Y. Liu, J. Loe, P. Luszczek, P. Nayak, S. Pranesh, S. Rajamanickam, T. Ribizel, B. Smith, K. Swirydowicz, S. Thomas, S. Tomov, Y.M. Tsai, I. Yamazaki, U.M. Yang, 2020
67. R.C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Martin, Robert C (Prentice Hall, 2018). URL https://books.google.fr/books?id=8ngAkAEACAAJ
68. J.D. Cook. Floating point numbers are a leaky abstraction, 2009. URL https://www.johndcook.com/blog/2009/04/06/numbers-are-a-leaky-abstraction/
69. J. Spolsky. The Law of Leaky Abstractions, 2002. URL https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions/
70. W. Kahan. Beastly numbers, Jan. 1996. URL https://people.eecs.berkeley.edu/~wkahan/tests/numbeast.pdf
71. D. Monniaux, ACM Trans. Program. Lang. Syst. **30**(3), 12:1, May 2008. doi:10.1145/1353445.1353446